МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №6

Разработка веб-приложения на Spring Boot.

Основы тестирования

Выполнил: студент группы ___БВТ2402___

Голиков Михаил Вячеславович

Руководитель: _Мосева Марина Сергеевна_

Москва, 2077

Цель работы

Изучить основы тестирования в Java и Spring Boot-приложениях: научиться писать модульные тесты, проверять поведение отдельных компонентов приложения, изолировать зависимости с помощью mock-объектов, а также познакомиться с базовыми подходами к тестированию сервисного слоя и веб-приложений.

Индивидуальное задание

На основе выполненной лабораторной работы №5 (система уведомлений с Spring Security и JWT) разработайте комплекс тестов для проверки корректности работы всех слоёв приложения. Реализуйте модульные тесты для сервисного слоя (UserService, NotificationService, AuthService) с использованием JUnit 5 и Mockito. Для каждого сервиса напишите как позитивные сценарии (успешное создание, получение, обновление, удаление), так и негативные (обработка ошибок, исключения при отсутствии данных). Используйте @Mock для изоляции зависимостей (репозиториев, мапперов, PasswordEncoder) и @InjectMocks для тестируемых сервисов. Примените ArgumentCaptor для проверки передаваемых в репозиторий объектов. Добавьте тест с использованием spy() для демонстрации частичного мокирования реальных объектов. Напишите веб-слой тесты для NotificationController с помощью @WebMvcTest и MockMvc, проверяющие корректность HTTP-ответов, статусов и преобразование JSON. Убедитесь, что все тесты проходят успешно, а при намеренном изменении ожидаемого значения тест падает. Продемонстрируйте выполнение тестов в среде разработки (IntelliJ IDEA) с зелёной и красной индикацией результатов.

Основная часть

В ходе выполнения лабораторной работы были последовательно реализованы все этапы для изучения тестирования в Spring Boot с использованием JUnit 5, Mockito и Spring Boot Test.

Первым шагом была проверка наличия зависимости spring-boot-starter-test в файле pom.xml. Эта зависимость подключает JUnit, Mockito, AssertJ, Hamcrest и другие полезные библиотеки для тестирования. В проекте она уже присутствовала, поэтому дополнительные настройки не потребовались.

В элементе 1 представлен код класса UserServiceTest. В этом тесте используются аннотации @ExtendWith(MockitoExtension.class) для интеграции Mockito с JUnit 5, @Mock для создания mock-объектов UserRepository и UserMapper, а также @InjectMocks для создания тестируемого сервиса с автоматическим внедрением моков.

В тесте getUserById_ShouldReturnUserDto_WhenUserExists проверяется успешный сценарий: когда пользователь существует в базе данных, сервис должен вернуть корректный UserDto. Используется when(userRepository.findById(...)).thenReturn(...) для имитации нахождения пользователя, а verify() проверяет, что метод репозитория действительно был вызван ровно один раз.

В негативном тесте getUserById_ShouldThrowException_WhenUserNotFound проверяется, что при отсутствии пользователя сервис выбрасывает исключение RuntimeException. Для этого используется assertThrows(RuntimeException.class, () -> {...}).

Тесты deleteUser проверяют как успешное удаление (с проверкой вызова userRepository.delete()), так и попытку удаления несуществующего пользователя.

```
package org.example.service;
```

```java
import org.example.model.dto.UserDto;
import org.example.model.entity.User;
import org.example.model.mapper.UserMapper;
import org.example.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDateTime;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @Mock
    private UserMapper userMapper;

    @InjectMocks
    private UserService userService;

    @Test
    void getUserById_ShouldReturnUserDto_WhenUserExists() {
        Long userId = 1L;
        User user = new User();
        user.setId(userId);
        user.setName("John Doe");
        user.setEmail("john@example.com");
        user.setPhone("+123456789");
        user.setCreatedAt(LocalDateTime.now());

        UserDto expectedDto = new UserDto();
        expectedDto.setName("John Doe");
        expectedDto.setEmail("john@example.com");
        expectedDto.setPhone("+123456789");

        when(userRepository.findById(userId)).thenReturn(Optional.of(user));
        when(userMapper.toDto(user)).thenReturn(expectedDto);

        UserDto result = userService.getUserById(userId);

        assertNotNull(result);
        assertEquals(expectedDto.getName(), result.getName());
        assertEquals(expectedDto.getEmail(), result.getEmail());
        assertEquals(expectedDto.getPhone(), result.getPhone());

        verify(userRepository, times(1)).findById(userId);
        verify(userMapper, times(1)).toDto(user);
    }
```

```
    @Test
    void getUserById_ShouldThrowException_WhenUserNotFound() {
        Long userId = 999L;
        when(userRepository.findById(userId)).thenReturn(Optional.empty());

        assertThrows(RuntimeException.class, () -> {
            userService.getUserById(userId);
        });

        verify(userRepository, times(1)).findById(userId);
        verify(userMapper, never()).toDto(any());
    }
    @Test
    void deleteUser_ShouldCallRepositoryDelete_WhenUserExists() {
        Long userId = 1L;
        User user = new User();
        user.setId(userId);

        when(userRepository.findById(userId)).thenReturn(Optional.of(user));
        doNothing().when(userRepository).delete(user);

        userService.deleteUser(userId);

        verify(userRepository, times(1)).findById(userId);
        verify(userRepository, times(1)).delete(user);
    }

    @Test
    void deleteUser_ShouldThrowException_WhenUserNotFound() {
        Long userId = 999L;
        when(userRepository.findById(userId)).thenReturn(Optional.empty());

        assertThrows(RuntimeException.class, () ->
userService.deleteUser(userId));
        verify(userRepository, never()).delete(any());
    }
}
```

Элемент 1 — UserServiceTest

В элементе 2 представлен код NotificationServiceTest. Особенность этого теста в том, что сервис зависит от двух репозиториев: NotificationRepository и UserRepository. Mockito позволяет легко создать моки для обоих и внедрить их в тестируемый сервис.

В тесте getNotificationById_ShouldReturnNotification_WhenExists проверяется получение уведомления по идентификатору. Mock-

объект notificationRepository настроен на возврат тестового уведомления при вызове findById(). Далее с помощью стандартных assertEquals проверяются все поля уведомления.

Закомментированный тест getNotificationById_ShouldThrowException_WhenNotificationNotFound (п оказан в элементе) демонстрирует, как можно было бы проверить выброс исключения при отсутствии уведомления. Это полезно для будущего расширения тестового покрытия.

```
package org.example.service;

import org.example.model.entity.Notification;
import org.example.model.entity.User;
import org.example.model.enums.NotificationChannel;
import org.example.model.enums.NotificationStatus;
import org.example.repository.NotificationRepository;
import org.example.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.time.LocalDateTime;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class NotificationServiceTest {

    @Mock
    private NotificationRepository notificationRepository;

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private NotificationService notificationService;

    @Test
    void getNotificationById_ShouldReturnNotification_WhenExists() {
        Long notificationId = 1L;
        User recipient = new User();
        recipient.setId(100L);

        Notification expectedNotification = new Notification();
```

```
        expectedNotification.setId(notificationId);
        expectedNotification.setTitle("Test Title");
        expectedNotification.setMessage("Test Message");
        expectedNotification.setChannel(NotificationChannel.EMAIL);
        expectedNotification.setStatus(NotificationStatus.CREATED);
        expectedNotification.setCreatedAt(LocalDateTime.now());
        expectedNotification.setRecipient(recipient);


when(notificationRepository.findById(notificationId)).thenReturn(Optional.of(
expectedNotification));

        Notification result =
notificationService.getNotificationById(notificationId);

        assertNotNull(result);
        assertEquals(notificationId, result.getId());
        assertEquals("Test Title", result.getTitle());
        assertEquals("Test Message", result.getMessage());
        assertEquals(NotificationChannel.EMAIL, result.getChannel());
        assertEquals(NotificationStatus.CREATED, result.getStatus());

        verify(notificationRepository, times(1)).findById(notificationId);
    }

//    @Test
//    void
getNotificationById_ShouldThrowException_WhenNotificationNotFound() {
//        // Arrange
//        Long notificationId = 999L;
//
when(notificationRepository.findById(notificationId)).thenReturn(Optional.emp
ty());
//
//        RuntimeException exception = assertThrows(RuntimeException.class,
() -> {
//            notificationService.getNotificationById(notificationId);
//        });
//
//        assertEquals(notificationId, exception.getMessage());
//
//        verify(notificationRepository, times(1)).findById(notificationId);
//    }
}
```

Элемент 2 — NotificationServiceTest

В элементе 3 представлен код AuthServiceTest. Этот тест особенно важен, так как AuthService содержит критическую бизнес-логику

регистрации: шифрование пароля, проверку уникальности email, установку роли и даты создания.

В тесте register_ShouldSaveUser_WhenEmailIsUnique используется ArgumentCaptor<User> — мощный инструмент Mockito, позволяющий "перехватить" объект, переданный в метод userRepository.save(), и проверить его поля после сохранения. Это необходимо, потому что метод register() сам создаёт объект User внутри себя, и мы не имеем к нему прямого доступа из теста.

```
package org.example.service;

import org.example.model.dto.RegisterRequest;
import org.example.model.entity.User;
import org.example.model.enums.UserRole;
import org.example.repository.UserRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class AuthServiceTest {

    @Mock
    private UserRepository userRepository;

    @Mock
    private PasswordEncoder passwordEncoder;

    @InjectMocks
    private AuthService authService;

    @Test
    void register_ShouldSaveUser_WhenEmailIsUnique() {
        RegisterRequest request = new RegisterRequest();
        request.setName("John Doe");
```

```
        request.setEmail("john@example.com");
        request.setPassword("password123");


when(userRepository.findByEmail("john@example.com")).thenReturn(Optional.empt
y());

when(passwordEncoder.encode("password123")).thenReturn("encodedPassword");
        when(userRepository.save(any(User.class))).thenAnswer(invocation ->
invocation.getArgument(0));

        authService.register(request);

        ArgumentCaptor<User> userCaptor =
ArgumentCaptor.forClass(User.class);
        verify(userRepository, times(1)).save(userCaptor.capture());

        User savedUser = userCaptor.getValue();
        assertEquals("John Doe", savedUser.getName());
        assertEquals("john@example.com", savedUser.getEmail());
        assertEquals("encodedPassword", savedUser.getPassword());
        assertEquals(UserRole.ROLE_USER, savedUser.getRole());
        assertNotNull(savedUser.getCreatedAt());

        verify(userRepository, times(1)).findByEmail("john@example.com");
        verify(passwordEncoder, times(1)).encode("password123");
    }

    @Test
    void register_ShouldThrowException_WhenEmailAlreadyExists() {
        RegisterRequest request = new RegisterRequest();
        request.setEmail("existing@example.com");

        User existingUser = new User();
        existingUser.setEmail("existing@example.com");


when(userRepository.findByEmail("existing@example.com")).thenReturn(Optional.
of(existingUser));

        RuntimeException exception = assertThrows(RuntimeException.class, ()
-> {
            authService.register(request);
        });

        assertEquals("Пользователь с email existing@example.com уже
существует", exception.getMessage());

        verify(userRepository, never()).save(any(User.class));
        verify(passwordEncoder, never()).encode(any());
    }

    @Test
    void registerAdmin_ShouldSaveAdminUser_WhenEmailIsUnique() {
        RegisterRequest request = new RegisterRequest();
        request.setName("Admin User");
```

```
        request.setEmail("admin@example.com");
        request.setPassword("admin123");


when(userRepository.findByEmail("admin@example.com")).thenReturn(Optional.emp
ty());

when(passwordEncoder.encode("admin123")).thenReturn("encodedAdminPassword");
        when(userRepository.save(any(User.class))).thenAnswer(invocation ->
invocation.getArgument(0));

        authService.registerAdmin(request);

        ArgumentCaptor<User> userCaptor =
ArgumentCaptor.forClass(User.class);
        verify(userRepository, times(1)).save(userCaptor.capture());

        User savedUser = userCaptor.getValue();
        assertEquals("Admin User", savedUser.getName());
        assertEquals("admin@example.com", savedUser.getEmail());
        assertEquals("encodedAdminPassword", savedUser.getPassword());
        assertEquals(UserRole.ROLE_ADMIN, savedUser.getRole());

        verify(userRepository, times(1)).findByEmail("admin@example.com");
    }

    @Test
    void registerAdmin_ShouldThrowException_WhenEmailAlreadyExists() {
        // Arrange
        RegisterRequest request = new RegisterRequest();
        request.setEmail("existing@example.com");

        User existingUser = new User();
        existingUser.setEmail("existing@example.com");


when(userRepository.findByEmail("existing@example.com")).thenReturn(Optional.
of(existingUser));

        assertThrows(RuntimeException.class, () -> {
            authService.registerAdmin(request);
        });

        verify(userRepository, never()).save(any(User.class));
    }
}
```

Элемент 3 — AuthServiceTest

В элементе 4 представлен класс SpyExampleTest, демонстрирующий работу spy() — механизма частичного мокирования реальных объектов.

В первом тесте testSpyWithList создаётся шпион на реальном ArrayList. Шпион вызывает настоящие методы add(), поэтому элементы реально добавляются в список. При этом с помощью verify() можно проверить, что эти вызовы действительно произошли. Во втором тесте testSpyWithStubbing показано, как можно переопределить поведение шпиона. С помощью doReturn(false).when(spyList).add(anyString()) мы заставляем шпиона возвращать false при любом вызове add(), хотя настоящий ArrayList всегда возвращает true при успешном добавлении. При этом сам элемент в список не добавляется.

```
package org.example.test;

import org.junit.jupiter.api.Test;
import org.mockito.Spy;

import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

class SpyExampleTest {

    @Test
    void testSpyWithList() {

        List<String> realList = new ArrayList<>();
        List<String> spyList = spy(realList);

        spyList.add("Element 1");
        spyList.add("Element 2");

        verify(spyList, times(1)).add("Element 1");
        verify(spyList, times(1)).add("Element 2");
        verify(spyList, times(2)).add(anyString());

        assertEquals(2, spyList.size());
        assertEquals("Element 1", spyList.get(0));
        assertEquals("Element 2", spyList.get(1));
    }

    @Test
    void testSpyWithStubbing() {

        List<String> list = new ArrayList<>();
        List<String> spyList = spy(list);

        doReturn(false).when(spyList).add(anyString());
```

```
        boolean result = spyList.add("Test");


        verify(spyList, times(1)).add("Test");
        assertEquals(false, result);
        assertEquals(0, spyList.size());
    }
}
```

Элемент 4 — SpyExampleTest

В элементе 5 представлен код NotificationControllerTest. Это пример веб-теста, который проверяет REST-контроллер на уровне HTTP-запросов без запуска всего приложения.

Аннотация @WebMvcTest(NotificationController.class) поднимает только веб-слой: контроллеры, преобразователи JSON, валидацию, но не загружает сервисы и репозитории. Зависимости контроллера (в данном случае NotificationService) подменяются с помощью @MockBean. Аннотация @AutoConfigureMockMvc(addFilters = false) отключает Spring Security для упрощения тестирования (чтобы не передавать JWT-токены).

MockMvc — это основной инструмент для отправки запросов и проверки ответов. Он имитирует HTTP-вызовы, но работает без реального порта и сетевого стека.

- perform(get(...)) — выполняет GET-запрос.
- andExpect(status().isOk()) — проверяет, что HTTP-статус 200 OK.
- andExpect(jsonPath("$.title").value(...)) — с помощью JsonPath проверяет конкретное поле в JSON-ответе.

Также реализованы тесты для всех основных эндпоинтов: получение всех уведомлений, удаление, фильтрация по статусу, каналу, получателю, комбинированные фильтры, создание и обновление.

```
package org.example.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
```

```
import org.example.model.dto.NotificationDto;
import org.example.model.entity.Notification;
import org.example.model.entity.User;
import org.example.model.enums.NotificationChannel;
import org.example.model.enums.NotificationStatus;
import org.example.security.JwtService;
import org.example.service.NotificationService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import java.time.LocalDateTime;
import java.util.Arrays;
import java.util.List;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.*;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(NotificationController.class)
@AutoConfigureMockMvc(addFilters = false)
class NotificationControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @MockBean
    private NotificationService notificationService;

    @MockBean
    private JwtService jwtService;

    @Test
    void getNotificationById_ShouldReturnNotification_WhenExists() throws
Exception {
        Long notificationId = 1L;
        Notification notification = createTestNotification(notificationId);


when(notificationService.getNotificationById(notificationId)).thenReturn(noti
fication);

        mockMvc.perform(get("/notifications/{id}", notificationId))
                .andExpect(status().isOk())
```

```
                .andExpect(jsonPath("$.title").value("Test Title"))
                .andExpect(jsonPath("$.message").value("Test Message"))
                .andExpect(jsonPath("$.channel").value("EMAIL"))
                .andExpect(jsonPath("$.status").value("CREATED"));

        verify(notificationService,
times(1)).getNotificationById(notificationId);
    }

    @Test
    void getAllNotifications_ShouldReturnListOfNotifications() throws
Exception {
        // Arrange
        List<Notification> notifications = Arrays.asList(
                createTestNotification(1L),
                createTestNotification(2L)
        );


when(notificationService.getAllNotifications()).thenReturn(notifications);

        mockMvc.perform(get("/notifications/all"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.length()").value(2))
                .andExpect(jsonPath("$[0].title").value("Test Title"))
                .andExpect(jsonPath("$[1].title").value("Test Title"));

        verify(notificationService, times(1)).getAllNotifications();
    }

    @Test
    void deleteNotification_ShouldReturnSuccessMessage() throws Exception {
        Long notificationId = 1L;

doNothing().when(notificationService).deleteNotification(notificationId);

        mockMvc.perform(delete("/notifications/{id}", notificationId))
                .andExpect(status().isOk())
                .andExpect(content().string("Уведомление удалено"));

        verify(notificationService,
times(1)).deleteNotification(notificationId);
    }

    @Test
    void getByStatus_ShouldReturnNotificationsByStatus() throws Exception {
        NotificationStatus status = NotificationStatus.CREATED;
        List<Notification> notifications =
Arrays.asList(createTestNotification(1L));


when(notificationService.getNotificationsByStatus(status)).thenReturn(notific
ations);

        mockMvc.perform(get("/notifications/status/{status}", status))
                .andExpect(status().isOk())
```

```
                .andExpect(jsonPath("$.length()").value(1));

        verify(notificationService,
times(1)).getNotificationsByStatus(status);
    }

    @Test
    void getByChannel_ShouldReturnNotificationsByChannel() throws Exception {
        NotificationChannel channel = NotificationChannel.EMAIL;
        List<Notification> notifications =
Arrays.asList(createTestNotification(1L));


when(notificationService.getNotificationsByChannel(channel)).thenReturn(notif
ications);

        mockMvc.perform(get("/notifications/channel/{channel}", channel))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.length()").value(1));

        verify(notificationService,
times(1)).getNotificationsByChannel(channel);
    }

    @Test
    void getByRecipientId_ShouldReturnNotificationsByRecipient() throws
Exception {
        Long recipientId = 100L;
        List<Notification> notifications =
Arrays.asList(createTestNotification(1L));


when(notificationService.getNotificationsByRecipientId(recipientId)).thenRetu
rn(notifications);

        mockMvc.perform(get("/notifications/recipient/{recipientId}",
recipientId))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.length()").value(1));

        verify(notificationService,
times(1)).getNotificationsByRecipientId(recipientId);
    }

    @Test
    void getByStatusAndChannel_ShouldReturnFilteredNotifications() throws
Exception {
        NotificationStatus status = NotificationStatus.SENT;
        NotificationChannel channel = NotificationChannel.SMS;
        List<Notification> notifications =
Arrays.asList(createTestNotification(1L));

        when(notificationService.getNotificationsByStatusAndChannel(status,
channel)).thenReturn(notifications);
```

```
mockMvc.perform(get("/notifications/statusandchannel/{status}/{channel}",
status, channel))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.length()").value(1));

        verify(notificationService,
times(1)).getNotificationsByStatusAndChannel(status, channel);
    }

    @Test
    void getByRecipientIdAndStatus_ShouldReturnFilteredNotifications() throws
Exception {
        Long recipientId = 100L;
        NotificationStatus status = NotificationStatus.CREATED;
        List<Notification> notifications =
Arrays.asList(createTestNotification(1L));


when(notificationService.getNotificationsByRecipientIdAndStatus(recipientId,
status)).thenReturn(notifications);


mockMvc.perform(get("/notifications/recipientidandstatus/{recipientId}/{statu
s}", recipientId, status))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.length()").value(1));

        verify(notificationService,
times(1)).getNotificationsByRecipientIdAndStatus(recipientId, status);
    }

    @Test
    void createNotification_ShouldReturnCreatedNotification() throws
Exception {
        NotificationDto requestDto = NotificationDto.builder()
                .title("New Notification")
                .message("Test Message")
                .channel(NotificationChannel.EMAIL)
                .recipientId(100L)
                .build();

        Notification notification = createTestNotification(1L);


when(notificationService.createNotification(any(NotificationDto.class))).then
Return(notification);

        mockMvc.perform(post("/notifications/add")
                        .contentType(MediaType.APPLICATION_JSON)

.content(objectMapper.writeValueAsString(requestDto)))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.title").value("Test Title"));

        verify(notificationService,
times(1)).createNotification(any(NotificationDto.class));
```

```
    }

    @Test
    void updateNotification_ShouldReturnUpdatedNotification() throws
Exception {
        Long notificationId = 1L;
        NotificationDto requestDto = NotificationDto.builder()
                .title("Updated Title")
                .message("Updated Message")
                .channel(NotificationChannel.SMS)
                .status(NotificationStatus.SENT)
                .recipientId(100L)
                .build();

        Notification updatedNotification =
createTestNotification(notificationId);

        when(notificationService.updateNotification(eq(notificationId),
any(NotificationDto.class)))
                .thenReturn(updatedNotification);

        mockMvc.perform(put("/notifications/{id}", notificationId)
                        .contentType(MediaType.APPLICATION_JSON)

.content(objectMapper.writeValueAsString(requestDto)))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.title").value("Test Title"));

        verify(notificationService,
times(1)).updateNotification(eq(notificationId), any(NotificationDto.class));
    }

    private Notification createTestNotification(Long id) {
        User recipient = new User();
        recipient.setId(100L);
        recipient.setName("Test User");
        recipient.setEmail("test@example.com");

        Notification notification = new Notification();
        notification.setId(id);
        notification.setTitle("Test Title");
        notification.setMessage("Test Message");
        notification.setChannel(NotificationChannel.EMAIL);
        notification.setStatus(NotificationStatus.CREATED);
        notification.setCreatedAt(LocalDateTime.now());
        notification.setRecipient(recipient);

        return notification;
    }
}
```

Элемент 5 — NotificationControllerTest

На изображении 6 показан результат успешного выполнения всех тестов в среде разработки IntelliJ IDEA. Зелёная галочка рядом с каждым тестом и зелёная полоса прогресса свидетельствуют о том, что:

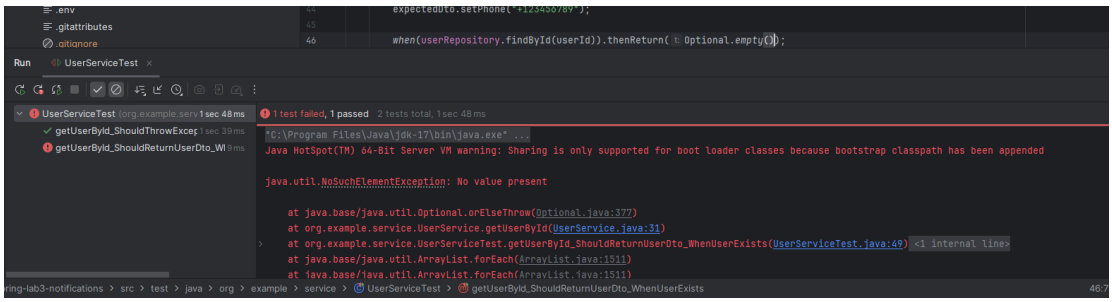
- все assertions прошли проверку;
- все ожидаемые исключения были выброшены (там, где это требовалось);
- все верификации вызовов mock-объектов совпали с реальным количеством вызовов.

В консоли вывода нет красных сообщений об ошибках. Это означает, что тестируемый код работает корректно в соответствии с заложенными в тесты ожиданиями.

Элемент 6 — Успешное выполнение теста

На изображении 7 показан результат выполнения теста после намеренного изменения ожидаемого значения. В одном из assertions было изменено ожидаемое значение (например, вместо "Test Title" ожидалось "Wrong Title"). В результате тест упал, и IntelliJ IDEA отобразила:

- красную полосу прогресса вместо зелёной;
- список провалившихся тестов;
- подробное сообщение об ошибке с указанием, какое именно assertion не сработало и какие значения сравнивались.



Элемент 7 — Неуспешное выполнение теста

Заключение

В ходе выполнения лабораторной работы были успешно освоены ключевые аспекты тестирования в Spring Boot-приложениях с использованием JUnit 5 и Mockito. На примере системы уведомлений, разработанной в предыдущих лабораторных работах, был реализован комплекс модульных и веб-тестов, покрывающих сервисный и контроллерный слои.

Github: https://github.com/Prototype721/Java